Highlights

R4Tun: LLM-guided adaptive segmental tunnel lining segmentation in point clouds

Xinghui Tao,Guangming Wang,Jelena Ninić,Brian Sheil

- Expert-tuned segmentation of segmental tunnel linings degrades when tunnel conditions depart from the reference setting.
- R4Tun supplements an expert-designed pipeline with LLM-guided stage agents that adapt parameters using three context components (memory, state, knowledge).
- Evaluated on 30 Seg2Tunnel subsets (13 regular, 17 complex) across three LLMs (Opus-4.6, GPT-5.4, Gemini-3-Flash).
- Mean mIoU increased by 110.7%–128.0% across the LLMs; intermediate pipeline state was the main context contributor.
- In its current form, R4Tun is most useful as an automated adaptation tool for tunnels close to the reference configuration, potentially reducing per-tunnel expert re-tuning.

R4Tun: LLM-guided adaptive segmental tunnel lining segmentation in point clouds

Xinghui Tao[a], Guangming Wang[a,*], Jelena Ninić[b] and Brian Sheil[a]

[a]Construction Engineering, University of Cambridge, Trumpington Street, Cambridge, CB2 1PZ, United Kingdom
[b]Department of Engineering, Durham University, Stockton Road, Durham, DH1 3LE, United Kingdom

ARTICLE INFO

Keywords:
Segmental tunnel lining
Point cloud segmentation
Tunnel inspection
Large language models
Parameter adaptation
Multi-agent systems

ABSTRACT

Automated inspection of segmental tunnel linings requires adaptive segmentation from 3D point clouds, yet expert-tuned pipelines often degrade when tunnel conditions vary. This paper presents R4Tun, a large language model (LLM)-driven adaptation framework that extends an expert-designed pipeline (SAM4Tun) with bounded parameter tuning informed by structured context (memory, state, and knowledge). Evaluated on 30 selected Seg2Tunnel subsets (13 regular, 17 complex) across three LLMs, this design increased mean Intersection-over-Union (mIoU) by at least 110.7% and 26.8% Overall Accuracy (OA). On complex tunnels, mIoU increased by 323.1%–361.7% (from 0.04 to 0.18–0.19). Across 270 (30 tunnels × 3 LLMs × 3 context settings) adapted runs, the LLMs showed similar parameter-adjustment trends and consistently adjusted a shared set of critical parameters. These results suggest that LLM-guided adaptation with structured context can help a fixed segmentation pipeline handle varying tunnel conditions and reduce per-tunnel expert re-tuning, particularly for tunnels close to the reference configuration.

1. Introduction

Automated inspection of segmental tunnel linings is required to support structural health assessment and long-term operational safety, given the scale, frequency, and access constraints of modern tunnel networks [1]. In practice, engineers routinely encounter projects in which tunnel geometry, lining properties, and data acquisition settings vary [2]. Furthermore, although modern laser scanning enables high-fidelity tunnel capture, the resulting measurements often contain mixed structural elements, occlusions, and noise, making direct extraction of lining components challenging [3, 4]. Consequently, segmentation methods need to be adaptive enough to handle this complexity and variability [5, 6].

Tunnel point-cloud segmentation methods in this study are grouped into three broad categories (see Fig. 1 and Section 2). Feature-engineering rules are interpretable but lack robustness under changed conditions [2]. Supervised deep-learning models generalise through data but require large labelled datasets and periodic retraining, while lacking auditability [7, 8, 9]. Foundation-model pipelines bypass annotation by using models pre-trained on large datasets [10, 11], but remain sensitive to expert-tuned processing parameters. Existing tunnel-segmentation approaches do not jointly provide strong adaptability to new conditions and an auditable adaptation process. Among foundation-model pipelines, SAM4Tun [12] combines point-cloud preprocessing with SAM-based prompting [11] and has shown strong performance under expert tuning. However, the pipeline is highly sensitive to its many processing parameters. When tunnel geometry or scanning conditions depart from the tuning reference, performance can degrade, and identifying which parameters require adjustment demands repeated expert intervention.

Given that recent LLMs can follow multi-step instructions over structured inputs, parameter adaptation can be formulated as a diagnostic task in which the model receives context and proposes bounded adjustments. LLMs have been applied to engineering workflows including chemical synthesis planning [13], multi-agent coordination for complex project tasks [14, 15, 16], and manufacturing decision support [17]. However, these studies do not address parameter adaptation in infrastructure inspection pipelines. In this study, we propose R4Tun, an LLM-guided adaptation system that extends the SAM4Tun pipeline by adjusting stage parameters in response to changing tunnel conditions. The framework provides each agent with three context components (memory, state, and knowledge), from which it produces bounded parameter updates via stepwise prompting. We evaluate whether this contextual information improves segmentation adaptability across both regular and complex tunnels, and whether the resulting adaptation behaviour is consistent across independent LLMs. To isolate the contribution of the LLM beyond deterministic tuning, we also benchmark R4Tun against a rule-based adaptation derived from the same per-stage knowledge documents.

This paper makes the following contributions:

1. A framework design in which LLM agents adapt the parameters of the SAM4Tun pipeline using a structured context: memory of the reference configuration, state from intermediate pipeline outputs, and a knowledge document of tunnel-category-specific configuration shared across all tunnels.
2. Empirical evidence from cumulative ablation across 30 tunnels suggesting that intermediate pipeline state is an important contributor to adaptation, while the

*Corresponding author
gw462@cam.ac.uk (G. Wang)
ORCID(s):

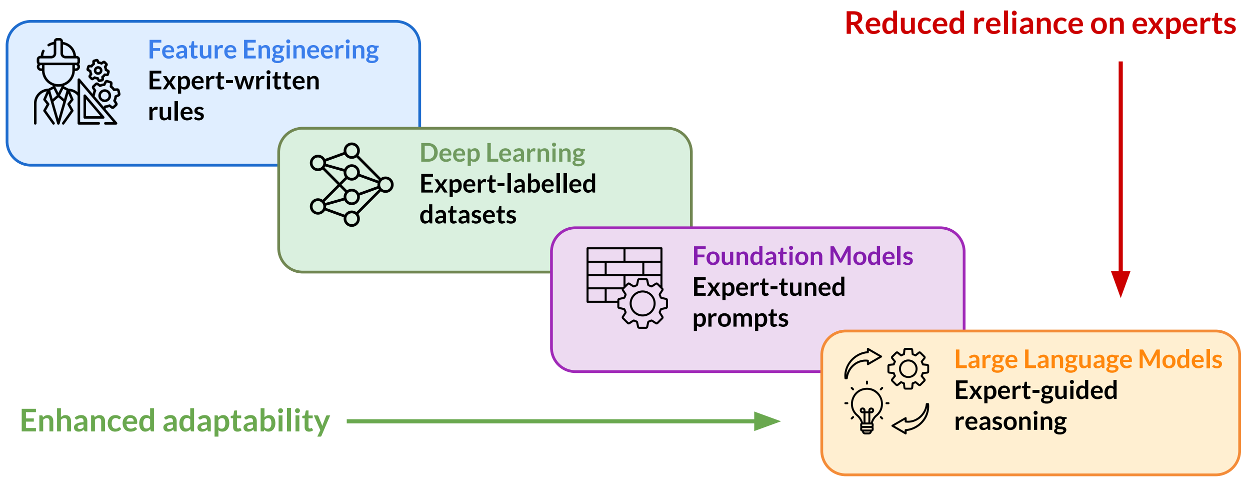


Figure 1: Automated segmentation approaches arranged by their adaptation mechanisms. R4Tun extends the foundation-model pipeline by adding LLM-guided parameter adaptation constrained by expert-guided reasoning.

knowledge component adds a smaller increment concentrated on the complex-category.

3. Cross-LLM validation showing that three independent LLMs (Opus-4.6, GPT-5.4, Gemini-3-Flash) follow similar adaptation patterns on a consistent subset of critical parameters.
4. Evidence that the observed mIoU increase cannot be reproduced by a deterministic, rule-based adaptation, and is therefore consistent with an additional contribution from the LLM-based adaptation process.

The rest of this paper is organised as follows. Section 2 reviews related work. Section 3 presents methodology, including the R4Tun architecture, dataset, experimental design, and evaluation. Section 4 reports results. Section 5 discusses findings, implications, and limitations. Section 6 concludes.

2. Related work

2.1. Feature engineering versus deep learning

Tunnel point-cloud segmentation has advanced through two generations of methods, which trade off between auditability and adaptability. Feature-engineering approaches encode domain knowledge into deterministic geometric rules: centreline fitting via RANSAC [18], density-based surface clustering [19], and point-sampled surface processing (e.g., simplification and resampling) [20]. Those methods remain widely used in safety-critical inspection because their logic is explicit and auditable [2]. However, each rule embeds assumptions about tunnel geometry and point-cloud characteristics (diameter, ring spacing, joint pattern, point density), and changing any of these requires manual reconfiguration by a domain expert. Supervised deep-learning methods, such as PointNet++ [21], RandLA-Net [22], Mask3D [23], and TD3D [24], learn features directly from annotated point clouds and can generalise across similar conditions present in the training set. Yet they require large labelled tunnel datasets that remain scarce [9], demand retraining for new tunnel conditions, and provide no mechanism for an engineer to trace or override a segmentation decision [8].

Therefore, for inspection operators, deploying to a new tunnel type requires either re-engineering rules, collecting new training data, or accepting degraded performance without a systematic diagnostic mechanism.

2.2. Foundation-model pipelines

To the best of our knowledge, direct 3D point-cloud segmentation of infrastructure at tunnel scene scale remains an open challenge for current foundation models. Existing 3D foundation models target common object recognition on curated datasets and do not transfer to large, noisy tunnel point clouds [25, 26, 27, 28]. As a result, practical tunnel pipelines often project 3D data into 2D representations to leverage mature vision foundation models pretrained on large-scale image datasets, thereby bypassing the annotation bottleneck [29, 30]. SAM [11] performs class-agnostic segmentation from spatial prompts without task-specific training, and extensions such as SAM2 [31] for temporal consistency and SEEM [32] for natural-language-guided segmentation are broadening prompt-based segmentation further. These models have been adopted in infrastructure contexts including crack detection [33] and Scan-to-BIM workflows [34, 35].

For tunnel linings, SAM4Tun [12] combines geometric preprocessing (unfolding, denoising, enhancing) with foundation model 2D segmentation, achieving strong segmentation without training data. However, this pipeline depends on numerous stage-specific parameters that encode assumptions about the reference tunnel's diameter, ring length, point density, and joint patterns. When a new tunnel departs from the reference, these parameters become misspecified and the pipeline degrades silently, without signalling which parameters fail or why. This sensitivity is not unique to

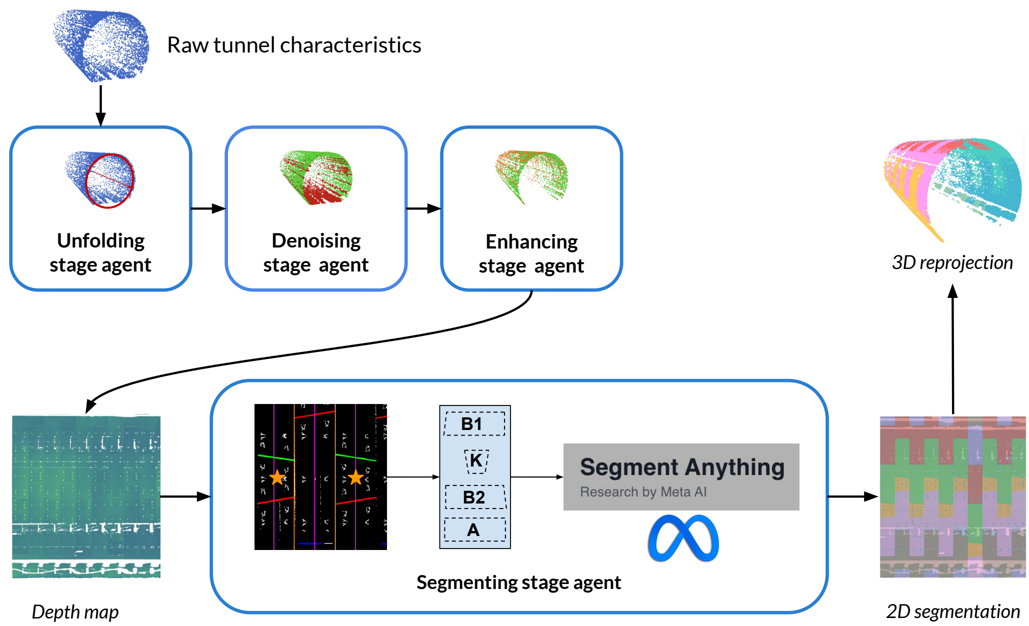


Figure 2: The R4Tun multi-agent architecture for tunnel segmentation. Four stage agents (unfolding, denoising, enhancing, and segmenting) adapt their parameters through LLM reasoning to generate segmentation.

SAM4Tun: most multi-stage pipelines that chain domain-specific preprocessing with a foundation model tend to inherit it. While foundation models have reduced reliance on labelled data, they have made the pipeline's dependence on hand-tuned parameters more explicit, without removing the need for expert parameter tuning.

2.3. LLM reasoning

Recent LLM advances have introduced explicit reasoning capabilities relevant to the parameter-adaptation problem. Reasoning-oriented training [36, 37, 38, 39, 40, 41] produces models capable of decomposing complex problems and self-verifying intermediate steps. Chain-of-thought (CoT) reasoning [42, 43] enables step-by-step logical traces. Multi-agent architectures coordinate specialised roles through shared context [15, 14, 16, 44, 45], and context engineering influences reasoning quality through the careful design of information provided to models [46, 47, 48].

In tunnelling segmentation, however, no prior work has applied LLM-based reasoning to a challenge in expert-designed pipelines: re-tuning a parameter-sensitive system to new conditions while preserving the engineer's ability to inspect and override each decision. We therefore introduce an LLM-based reasoning framework that retunes parameter-sensitive pipelines per tunnel using expert-guided context and explicit rationale. Meanwhile, whether LLM reasoning produces gains beyond what hand-coded rules can deliver from the same knowledge text remains an open question. Our experimental design therefore tests this directly by including a deterministic rule-based adaptation of those documents as a Non-LLM control.

3. Methodology

3.1. Task definition

The task is semantic segmentation of segmental tunnel linings from terrestrial laser scanning (TLS) point clouds. Given a raw point cloud of a tunnel section, the goal is to assign each point a structural label: background, key block (K), base blocks (B1, B2), and adjacent blocks (A1–A3), with an additional A4 class for 7-segment complex tunnels. The unit of analysis is a tunnel subset spanning multiple rings selected from the Seg2Tunnel benchmark.

A key limitation is that the fixed configuration of the open-source SAM4Tun achieves a mean mIoU of 0.29 on regular tunnels, but only 0.04 on complex tunnels whose geometry departs from the tuning reference. As noted above, the pipeline provides no diagnostic feedback when parameters become misspecified.

3.2. R4Tun

R4Tun supplements the SAM4Tun pipeline [12] with an LLM-guided adaptation layer that adjusts parameters per tunnel without modifying the underlying algorithms (Fig. 2). Given a new tunnel point cloud, a characteriser first extracts raw geometric properties (diameter, density, coordinate ranges; full field list in Appendix B). Each of the four stages (unfolding, denoising, enhancing, and segmenting) has a dedicated LLM agent that adapts its parameters. In the segmenting stage, the adapted prompts are passed to SAM, and the resulting 2D masks are reprojected onto the 3D point cloud without further LLM intervention.

Stage 1-Unfolding (Fig. 3) establishes a cylindrical coordinate system for the tunnel. It slices the point cloud at

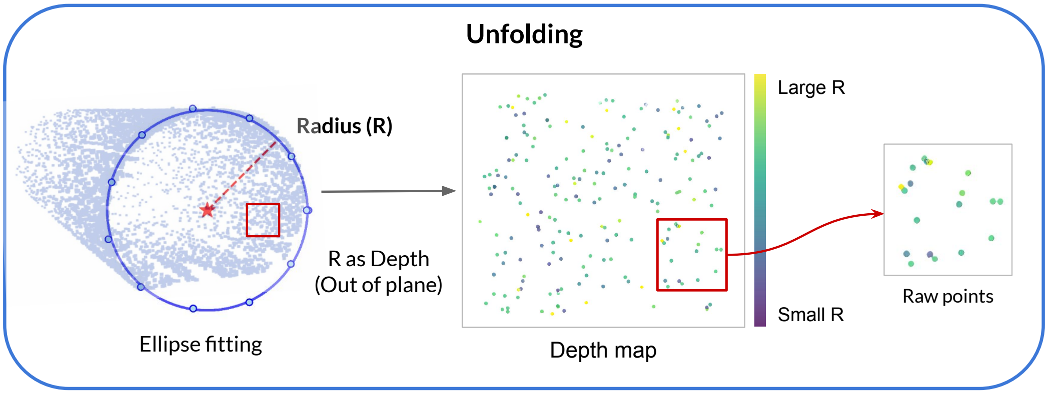


Figure 3: Stage 1-Unfolding. Left: fitted tunnel cross-section used to determine the reference centre and radius for cylindrical unfolding. Centre: 2D panoramic depth map derived from the 3D point cloud. Right: zoomed region of the unfolded map showing radial-distance encoding.

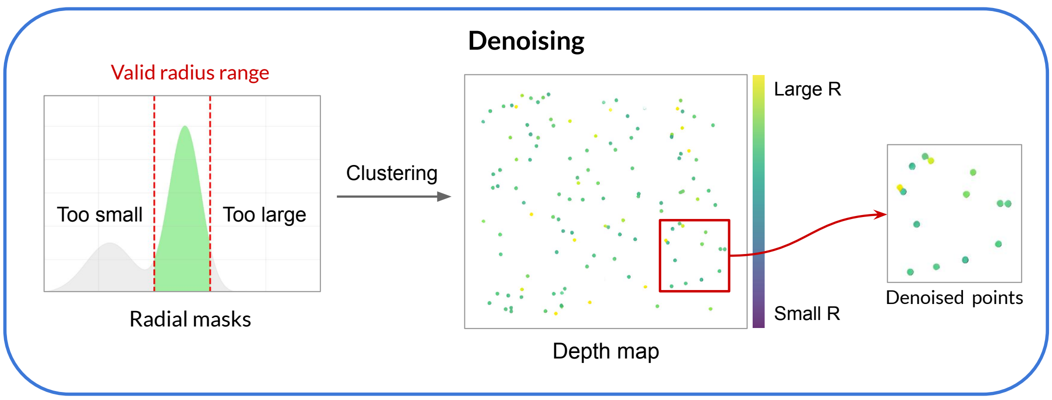


Figure 4: Stage 2-Denoising. Left: radial-distance distribution used to filter points outside the expected tunnel lining surface. Centre: unfolded 2D depth map after filtering, showing the cleaned point distribution. Right: zoomed region of the denoised map.

regular longitudinal intervals, fits an ellipse to each cross-section via RANSAC [18], and selects the model with the highest inlier support. The per-slice ellipse centres are interpolated into a smooth centreline that defines the cylindrical frame (r, θ, h). The 3D tunnel surface is then projected into a 2D panoramic image in which each pixel encodes the radial distance to the centreline, producing a stable depth representation for downstream filtering and segmentation.

Stage 2-Denoising (Fig. 4) removes non-structural artefacts (rails, cables, and scattered points) that would otherwise interfere with segmentation. The algorithm applies grid-based radial-density filtering inspired by the DBSCAN clustering principle [19], grouping points by local density in cylindrical coordinates. Dense, connected regions are retained as tunnel surfaces while isolated points are rejected as noise. A pair of radial masks (R_{low}, R_{high}) constrains filtering to the expected tunnel radius, preserving joint boundaries and ring edges while discarding outliers beyond the lining surface. The result is a clean, continuous surface representation suitable for curvature analysis and interpolation in the next stage.

Stage 3-Enhancing (Fig. 5) improves geometric continuity and surface completeness before projection into the 2D depth map. Local curvature is estimated on neighbourhood points to guide point insertion: new points are placed between neighbours whose curvature difference is small (smooth panel regions), while high-curvature areas (joint boundaries and ring edges) are preserved intact. A three-stage progressive upsampling scheme inserts additional points at progressively finer resolutions to maintain surface continuity without blurring structural boundaries. Pixel-level interpolation then refines outlier points at joint locations. The enhanced surface is projected into a panoramic depth map that balances smooth panel regions with sharply defined joint boundaries.

Stage 4-Segmenting (Fig. 6) combines boundary detection and SAM segmentation into a single workflow. First, a Hough-transform-based detector [49] extracts linear features from the enhanced depth map that correspond to ring joint boundaries. Detected lines are filtered by orientation (oblique, horizontal, vertical) with separate Hough thresholds and merged when closer than a distance tolerance. The confirmed boundaries are then used to construct template-based prompts for key, base, and adjacent segments, which are passed to SAM [11]. SAM produces 2D segment masks on the panoramic image, which are reprojected into 3D point labels via the stored pixel-to-point mapping from Stage 1.

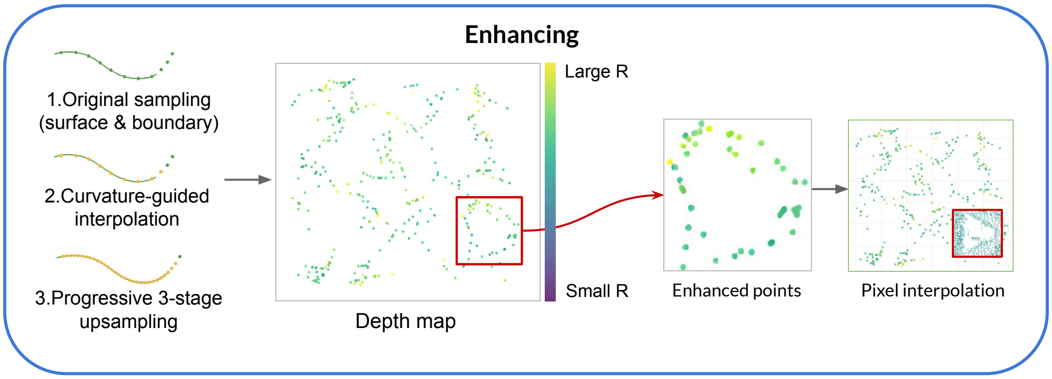


Figure 5: Stage 3-Enhancing. Left: curvature-guided surface and boundary interpolation along the tunnel lining, showing original sampling, curvature-guided insertion, and progressive three-stage upsampling. Centre: unfolded 2D depth map after enhancement, where inserted points improve geometric continuity; the red box highlights a local region of interest. Right: zoomed views of the highlighted region, showing added points and pixel-level interpolation.

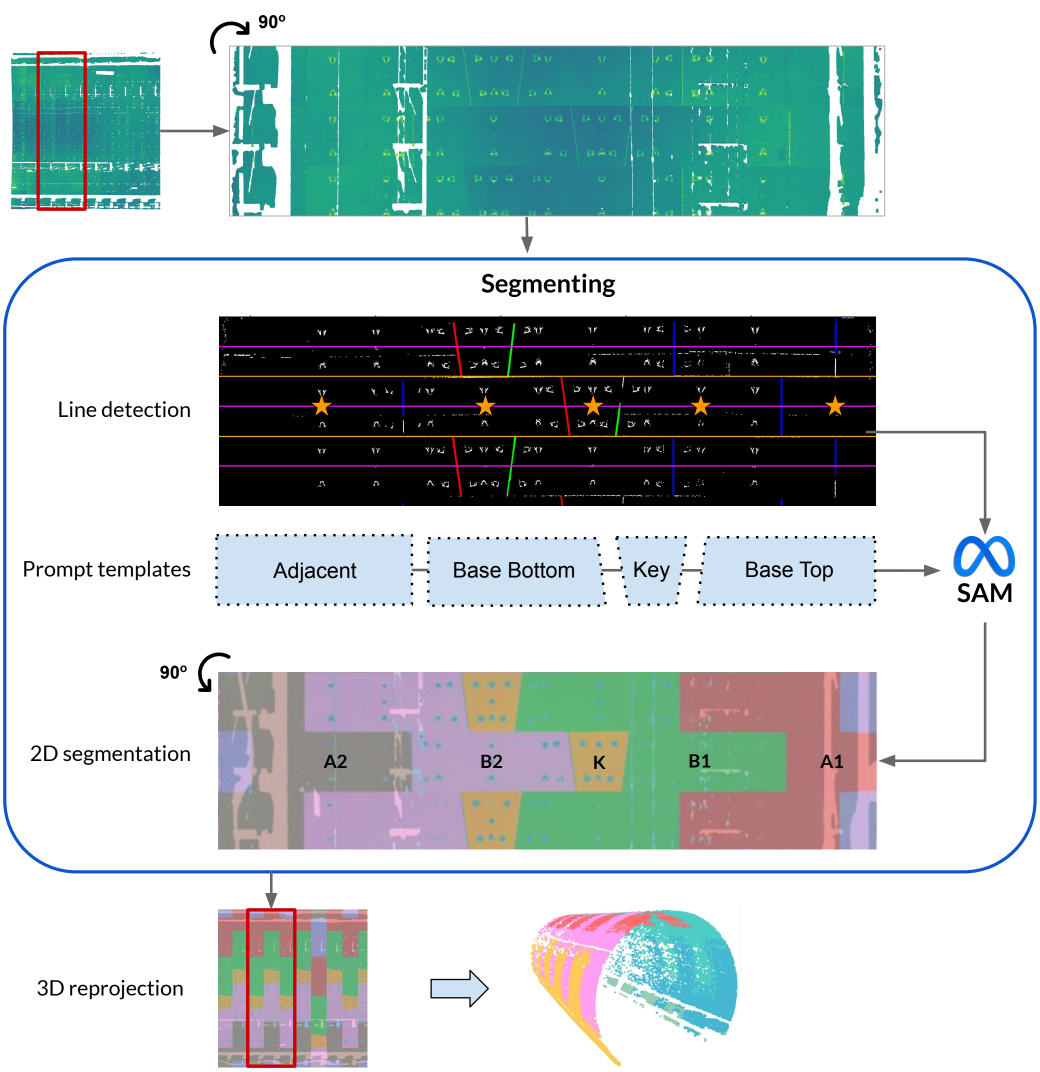


Figure 6: Stage 4-Segmenting: Hough-transform line detection [49] to identify ring boundaries, constructs template-based prompts, and feeds them to SAM [11], which produces 2D segment masks that are reprojected into 3D. (Note: rotation is shown for visualisation purposes only.)

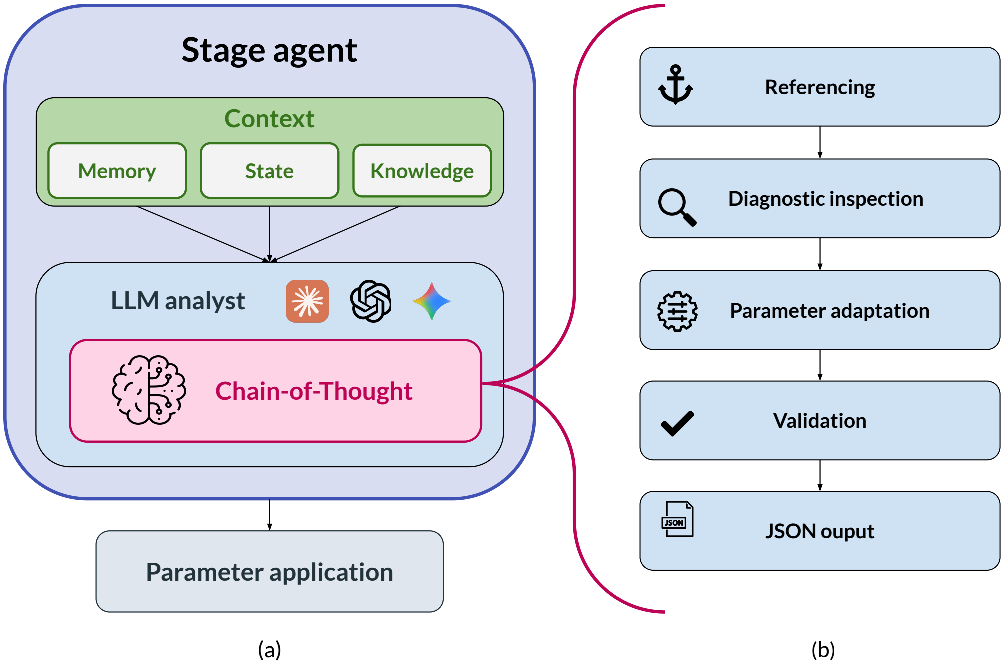


Figure 7: The left column (a) shows R4Tun stage agent architecture. Each agent maintains a shared and cumulative context that informs an LLM analyst component performing CoT reasoning. The right column (b) shows the five CoT phases: (1) Referencing, (2) Diagnostic inspection, (3) Parameter adaptation, (4) Validation, and (5) JSON output.

The pipeline contains 81 tuneable processing parameters, spanning geometric defaults, filtering and interpolation settings, and boundary-detection thresholds. All parameters were expert-tuned on a reference tunnel (diameter 5.60 m, density 2,466 pts/m^3, mIoU = 0.531); full parameter tables are provided in Appendix A. In this study, the underlying pipeline stages remain fixed across all conditions; only the parameter values change. The SAM4Tun baseline applies this single expert-tuned configuration uniformly to all 30 tunnels without per-tunnel adaptation, serving as the static baseline against which LLM-guided adaptation is measured.

3.3. Agent design

Each of the four stage agents is a self-contained unit comprising (i) a *context* (Section 3.3.1; Fig. 7a) and (ii) a Python *analyst* that constructs the full LLM prompt and parses the returned JSON. For each agent-driven stage, the analyst assembles a prompt from the current context level, calls one of the selected LLM APIs (Opus-4.6, GPT-5.4, or Gemini-3-Flash) to follow a five-step CoT protocol (Section 3.3.2; Fig. 7b): referencing, diagnostic inspection, parameter adaptation, validation, and JSON output. Each agent returns a single schema-conformant parameter JSON, which the corresponding pipeline stage executes unchanged. A characteriser plugin then extracts statistics from the stage output, updating the cumulative state available to subsequent stages. Each stage's parameters are adapted from the expert-tuned reference configuration together with this cumulative state; the stage scripts and evaluation code are identical across all ablation conditions, with only the parameter JSONs changing. Because every parameter change is logged alongside a generated textual rationale, engineers can review and, where necessary, override adaptation decisions. A worked example is provided in Appendix E. After the final segmentation, per-point labels are evaluated against ground truth using mean Intersection over Union (mIoU) as the primary metric.

3.3.1. Context design

Each stage agent receives structured context comprising three components: memory, state and knowledge (Fig. 7a). A worked example of the three components for the denoising agent is given in Appendix D.

Memory stores the reference tunnel's characteristics (a JSON file containing geometry, point density, coordinate ranges, nearest-neighbour distances) alongside the expert-tuned reference parameters. The agent compares the current tunnel's characteristics against this reference to quantify deviation and adjust its parameters via CoT reasoning (Section 3.3.2). Memory is static: it does not change between stages.

State captures cumulative geometric and statistical properties after each pipeline stage executes. These JSON summaries are injected into subsequent agents' prompts, and the state grows as stages execute. Importantly, state alone does not prescribe parameter changes: the numeric summaries of the actual point distribution after upstream processing must be interpreted in the context of parameter semantics and tunnel conditions before they can inform parameter updates.

Knowledge supplies domain-specific guidance in human-readable markdown documents. Each stage has its own knowledge document covering: tunnel-type taxonomy; parameter semantics and empirically validated ranges; and diagnostic rules linking characteristics to parameter adjustments. Knowledge is authored once and shared across all tunnels.

3.3.2. CoT design

CoT reasoning provides the analytical structure for each agent's decision-making (Fig. 7b). The agent follows a strict five-step protocol(Algorithm 1). A worked example of the full five-step trace is provided in Appendix E.

1. *Referencing:* The agent compares the current tunnel's characteristics against the stored reference to quantify deviation (e.g., +36% diameter increase).
2. *Diagnostic inspection:* The agent attributes the deviation to a plausible cause and identifies which specific tunable parameters are implicated. If signals conflict, the agent is instructed to prioritise *State* over *Memory*, as State reflects the actual geometric reality following upstream processing.
3. *Parameter adaptation:* The agent proposes updates for the implicated parameters. These updates are strictly bounded by the empirical ranges defined in the *Knowledge* component.
4. *Validation:* A self-correction step performs a consistency check to confirm that the proposed values satisfy logical constraints (e.g. a radius lower bound must be smaller than the corresponding upper bound). If a constraint is violated, the value is clipped to the nearest valid bound. This check-and-correction is executed as an LLM reasoning step condition, rather than a deterministic software routine, and therefore does not guarantee constraint satisfaction.
5. *JSON output:* The selected parameters and their reasoning trace are packaged into a single schema-conformant JSON object.

Notation. C_{target} denotes the raw characteristics of the current tunnel, while M stores the reference configuration ($C_{\text{ref}}, P_{\text{ref}}$). S is the cumulative pipeline state, and S_{current} is its most recent summary. K is a shared knowledge document that encodes parameter semantics, per-parameter admissible bounds B, and cross-parameter constraints R. The adaptation procedure computes a geometric deviation descriptor Δ_{geom}, selects an implicated parameter subset $P_{\text{implicated}}$, proposes an updated parameter map P_{proposed} (indexed as $P_{\text{proposed}}[p]$), and finally returns the schema-formatted JSON object P_{adapted}.

Algorithm 1 LLM-driven Parameter Adaptation via CoT

Require: Target tunnel raw characteristics C_{target}
Require: Memory M (reference characteristics C_{ref}, reference parameters P_{ref})
Require: State S (cumulative pipeline outputs / intermediate summaries)
Require: Knowledge K (parameter semantics, tunable bounds B, and cross-parameter constraints R)
Ensure: Adapted stage parameter JSON P_{adapted}

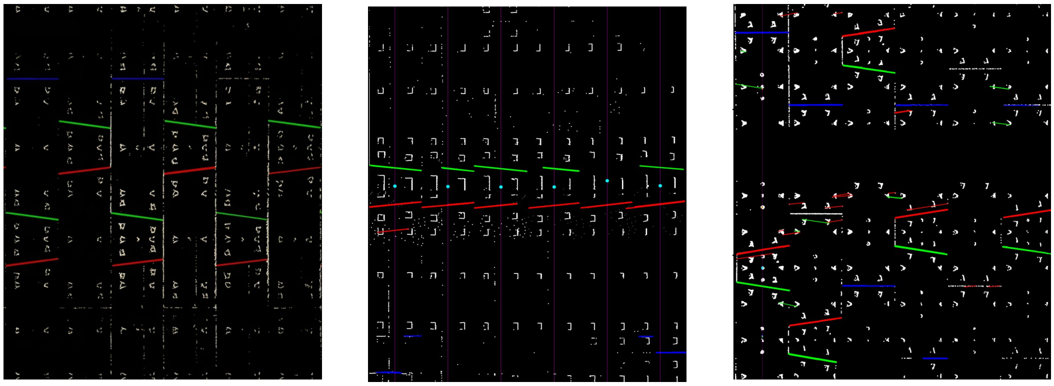
Phase 1: Referencing
$\Delta_{\text{geom}} \leftarrow \text{Compare}(C_{\text{target}}, C_{\text{ref}})$ ▷ Quantify deviation from reference
Phase 2: Diagnostic inspection
$S_{\text{current}} \leftarrow \text{ExtractLatest}(S)$ ▷ Most recent stage statistics
Reconciliation: If S_{current} conflicts with M, prioritise S_{current}
$P_{\text{implicated}} \leftarrow \text{IdentifyParameters}(\Delta_{\text{geom}}, S_{\text{current}}, K)$ ▷ Select parameters relevant to observed deviations
Phase 3: Parameter adaptation
$P_{\text{proposed}} \leftarrow \emptyset$
for each parameter $p \in P_{\text{implicated}}$ **do**
    $P_{\text{proposed}}[p] \leftarrow \text{Adjust}(p, \Delta_{\text{geom}}, S_{\text{current}}, K)$ ▷ Propose new value within B guided by K
end for
Phase 4: Validation
for each parameter $p \in P_{\text{proposed}}$ **do**
    $P_{\text{proposed}}[p] \leftarrow \text{SelfValidate}\big(P_{\text{proposed}}[p],\ p,\ K\big)$ ▷ Execute an LLM internal reasoning step conditioned on K (semantics, bounds spec B, constraints R).
end for
Phase 5: JSON output
$P_{\text{adapted}} \leftarrow \text{FormatAsJSON}(P_{\text{proposed}})$
return P_{adapted}

3.4. Experimental setup

3.4.1. Dataset

We selected 30 tunnel subsets from the Seg2Tunnel benchmark (Table 1). These subsets cover the main variations in segmental tunnel geometry (diameter, ring length, segment count, key-block layout, and point-density distribution), providing a controlled basis for evaluating adaptation across tunnel categories.

As shown in Fig. 8, staggered and continuous tunnels are grouped as "regular" ($n = 13$): they share the 5.60 m nominal diameter, 1.2 m ring spacing, six segments per ring, and, most importantly, regular patterns of key-block positions similar to the reference. Complex tunnels ($n = 17$) depart from that reference in multiple coupled ways: inner diameter increases by 34% (7.5 m), ring length by 50% (1.8 m), and each ring adds a seventh segment with a complex interleaved key-block arrangement. In addition, off-axis scanning yields non-uniform sampling, partial circumferential coverage, and angle-dependent range and incidence. Together, these differences alter both the geometry visible in the point cloud and the semantic targets the pipeline must recover. Consequently, a fixed parameter set cannot be assumed to transfer from the regular reference; these conditions constitute the primary stress test for whether R4Tun can adapt the same algorithmic pipeline to off-design tunnel conditions. Ground-truth labels



Regular (staggered) Regular (continuous) Complex

Figure 8: From left to right: regular (staggered) tunnels exhibit a repeating ring-to-ring pattern; regular (continuous) tunnels maintain a fixed segment order around each ring; complex tunnels show no consistent repeating pattern.

Table 1
Dataset properties.

	Regular (T1, T2, T3)		Complex (T4, T5)
Property	Staggered (T1, T2)	Continuous (T3)	
Inner diameter	5.60 m	5.60 m	7.5 m
Ring length	1.2 m	1.2 m	1.8 m
Segments/ring	6	6	7
Joint type	Staggered	Continuous	Complex interleaved
Key-block position	A ring-to-ring repeating pattern	A fixed ring-wise order	No repeating pattern
Scanning	Single-station	Multi-station	Single-station, offset
Eval. schema	6-class	6-class	7-class
Count	10 subsets	3 subsets	17 subsets

are per-point structural class annotations provided by the Seg2Tunnel benchmark; they are used for evaluation only and are not provided to the LLM or pipeline during adaptation.

3.4.2. *Experimental design*

The experiment follows a cumulative ablation design with four conditions, each adding one context component (Table 2). This design isolates the cumulative contribution of each component while maintaining a controlled comparison. Level 0 (SAM4Tun) is the fixed expert-tuned baseline with no LLM involvement. Levels 1–3 progressively provide the LLM with richer context, enabling the cumulative ablation to attribute improvements to specific components. A separate *Non-LLM* condition (level 0a in Table 2) is included as the Non-LLM adaptive control, allowing the LLMs' contribution to be read directly from the gap between the *Non-LLM* column and the LLM columns of Table 4.

3.5. Non-LLM adaptation

We codified the per-stage knowledge documents into a deterministic Python rule table as a non-LLM baseline. It maps characterisation fields (e.g., diameter, ring length, segments-per-ring, joint type, point density, and station configuration) to the 18 critical parameters adjusted by the LLMs (Table 7) using explicit "if… then…" rules. The

Table 2
Ablation conditions.

Level	Code	Condition	What the LLM sees
0	SAM4Tun	Baseline	Default parameters
0a	Non-LLM	Rule table	18 parameters set by lookup
1	m	Memory	Reference characteristics + parameters
2	m+s	Memory+State	+ intermediate pipeline outputs
3	m+s+k	Memory + State + Knowledge	+ domain knowledge

table and script are fixed for all 30 tunnels (regular and complex), with no evaluation-set tuning or subset-specific settings, isolating the value of LLM per-tunnel reasoning. The complete rule specification is provided in Appendix C.

To assess whether adaptation behaviour depends on a specific LLM, each condition was run with three independent LLMs (Opus-4.6, GPT-5.4, and Gemini-3-Flash) under identical prompts. Across models, the pipeline code, evaluation scripts, and prompt structure were held constant, while the context content varied by ablation level. We used

Table 3
LLM configuration.

Setting	Value
Models	Opus-4.6, GPT-5.4, Gemini-3-Flash
Max tokens	16,384
Temperature	0 (minimal stochasticity)
Timeout	300 s per call
Prompt format	Markdown with JSON code
Failure handling	JSON extraction fail → log & error

each API's default configuration (Table 3). Each of the three ablation conditions was run once per LLM for all 30 tunnels, yielding 3×3×30 = 270 pipeline executions plus 30 baseline runs with fixed parameters. Reported results should therefore be interpreted as single-run outcomes under vendor-default API settings; stochastic variability across repeated calls was not characterised. All three LLMs were accessed via their respective commercial APIs (Table 3) [50]. No model-specific prompt tuning was performed.

Pipeline execution used a single NVIDIA RTX 5060 GPU. Adapted parameter files and CoT reasoning traces were logged for all runs, enabling post-hoc sensitivity analysis (Section 3.5.2).

3.5.1. Evaluation metrics

Segmentation quality is measured primarily by mean Intersection-over-Union (mIoU). For class c,

$$\mathrm{IoU}_c = \frac{\mathrm{TP}_c}{\mathrm{TP}_c + \mathrm{FP}_c + \mathrm{FN}_c}, \qquad \mathrm{mIoU} = \frac{1}{C}\sum_{c=1}^{C} \mathrm{IoU}_c, \quad (1)$$

where C is the number of segmental blocks (classes) within one ring ($C = 6$ for regular tunnels; $C = 7$ for complex tunnels). As a complementary global metric we also report overall accuracy (OA),

$$\mathrm{OA} = \frac{\sum_{c=1}^{C} \mathrm{TP}_c}{N}, \quad (2)$$

where N is the total number of points. Because OA can be dominated by majority classes, we treat mIoU as the primary score. We also report per-class IoU to assess whether improvements are broad or concentrated in specific structural classes.

For each condition and LLM, we computed paired per-tunnel improvements as $\Delta_i = \mathrm{mIoU}_{\mathrm{condition},i} - \mathrm{mIoU}_{\mathrm{baseline},i}$, with $n = 13$ for regular tunnels, $n = 17$ for complex tunnels, and $n = 30$ overall. We summarise these paired improvements using three standard statistics: (i) two-sided paired t-test p-values ($\alpha = 0.05$) for statistical significance, (ii) paired Cohen's d for effect size, and (iii) bootstrap 95% confidence intervals (CIs) for the mean improvement. Both bootstrap 95 % CIs and paired t-test p-values are reported as complementary statistics, acknowledging that the two methods may occasionally diverge near the $\alpha = 0.05$ threshold.

3.5.2. Sensitivity analysis

To assess the robustness of adapted parameters, for each parameter, we report the *coefficient of variation* (CV):

$$\mathrm{CV} = \frac{s}{|\bar{v}|}, \quad (3)$$

where $\bar{v} = \frac{1}{N}\sum_{i=1}^{N} v_i$ and $s = \sqrt{\frac{1}{N-1}\sum_{i=1}^{N}(v_i - \bar{v})^2}$ are the mean and standard deviation of the adapted values $\{v_i\}_{i=1}^{N}$ across $N = 30$ tunnels. CV measures how much a parameter changes from tunnel to tunnel: (i) a high CV (≥ 0.06) identifies parameters that are *tunnel-responsive*; (ii) CV ≈ 0 indicates *baseline corrections* that take the same improved value on every tunnel regardless of geometry. We further examine *cross-LLM consistency*: for parameters that always trigger adaptation, we compare the adapted values, tunnel counts, and tunnel categories produced by each LLM independently.

4. Results

4.1. Overall performance

Table 4 summarises mean mIoU across conditions and LLMs. Under the full m+s+k design, overall mIoU rises from 0.15 (baseline) to 0.32–0.34 across the three LLMs (a relative increase of 110.7%–128.0%; $p < 0.0001$, paired Cohen's $d = 1.51$–1.95); overall OA rises from 0.41 to 0.52–0.55 (a relative increase of 26.8%–34.1%). The improvement holds across both tunnel families (Fig. 9). Regular tunnels improve from 0.29 to 0.50–0.54 (a relative increase of 70.0%–83.8%; effect sizes $d = 1.4$–2.2); regular-tunnel OA rises from 0.51 to 0.67–0.71 (a relative increase of 31.3%–39.2%). Complex tunnels, where the baseline drops to mIoU = 0.04 because several pipeline assumptions no longer match the tunnel conditions, improve to 0.18–0.19 (a relative increase of 323.1%–361.7%; $d = 1.2$–2.5), an absolute increase of 0.14–0.15 mIoU; complex-tunnel OA rises from 0.34 to 0.41–0.43 (a relative increase of 21.6%–28.2%).

Wall-clock runtime, API-call counts, and per-tunnel input/output token usage with the corresponding indicative USD cost for each LLM under m+s+k are reported in Appendix F (Table 15). Per-class IoU analysis (Appendix G) supports broad improvement across structural classes for regular tunnels and progressive recovery from near-zero baselines for complex tunnels. The full performance distribution is summarised in Appendix H.

Comparison against the Non-LLM adaptation. The Non-LLM rule-based adaptation (Table 4, *Non-LLM* column) raises overall mIoU from 0.15 to 0.20 (a relative increase of 34.2%; +0.051, $p = 0.0035$), indicating that part of the gain may reflect deterministic adaptation rather than the LLM-based step alone. The performance distribution varies markedly across tunnel categories: on regular tunnels, the Non-LLM adaptation and the static baseline are statistically indistinguishable (≈ 0.29; $p = 0.79$), which is likely due to the rule table reproducing the configuration that SAM4Tun was originally tuned for.

Table 4
Overall segmentation results ($n = 30$): mean mIoU and paired ΔmIoU vs. baseline with p-values, effect sizes, and bootstrap 95% CIs. Δ is paired against SAM4Tun. OA is reported as a secondary metric, shown as the first line in each condition block.

Condition	Statistic	Non-LLM	Opus-4.6	GPT-5.4	Gemini-3-Flash
Baseline (SAM4Tun)	Mean OA	0.411	0.411	0.411	0.411
	Mean mIoU	0.150	0.150	0.150	0.150
Non-LLM	Mean OA	0.407	—	—	—
	Mean mIoU	0.201	—	—	—
	Mean ΔmIoU	+0.051	—	—	—
	p-value	0.0035	—	—	—
	Cohen's d	0.58	—	—	—
	std (Δ)	0.088	—	—	—
m	Mean OA	—	0.397	0.428	0.443
	Mean mIoU	—	0.144	0.182	0.199
	Mean ΔmIoU	—	−0.006	+0.032	+0.049
	p-value	—	0.558	0.028	0.056
	Cohen's d	—	−0.11	0.42	0.36
	95% CI	—	[−0.027, 0.014]	[0.005, 0.058]	[0.006, 0.101]
m+s	Mean OA	—	0.535	0.520	0.525
	Mean mIoU	—	0.320	0.299	0.302
	Mean ΔmIoU	—	+0.170	+0.149	+0.152
	p-value	—	< 0.0001	< 0.0001	< 0.0001
	Cohen's d	—	1.77	1.32	1.46
	95% CI	—	[0.137, 0.204]	[0.110, 0.190]	[0.117, 0.189]
m+s+k	Mean OA	—	0.552	0.537	0.522
	Mean mIoU	—	**0.342**	**0.321**	**0.316**
	Mean ΔmIoU	—	+0.192	+0.171	+0.166
	p-value	—	< 0.0001	< 0.0001	< 0.0001
	Cohen's d	—	1.95	1.95	1.51
	95% CI	—	[0.158, 0.227]	[0.140, 0.203]	[0.126, 0.204]

On complex tunnels, the rules add +0.095 mIoU ($p = 0.0001$), a 225.2% increase over the complex-category static baseline (0.04), by switching tunnel diameter, ring spacing, segment count, and key-block layout to their large-tunnel specification (i.e., lining-design parameters typically available to on-site engineers). Relative to this rule-based baseline, the LLM (m+s+k) yields a further increase in mean mIoU from 0.137 to 0.178–0.194 (an additional +0.041 to +0.057, i.e., +30.1%–42.0% relative), improving 10 of 17 complex tunnels. The remaining seven cases (six rule wins and one tie) occur primarily in tunnels where the deterministic large-tunnel rules already provide a strong prior; in this low-mIoU regime, LLM updates exhibit higher cross-model variance and do not reliably outperform the rule baseline.

The gap is largest on the regular-category, where the non-LLM rule table does not improve beyond the SAM4Tun configuration (≈ 0.29), whereas the LLM-based adaptation raises mIoU to 0.50–0.54 (70.0%–83.8% relative). Under this experimental setup, the additional gain is therefore consistent with an LLM contribution beyond the deterministic control. On the complex-category, the rules recover roughly 60 % of the LLM gain (static: 0.04; rules: 0.137; LLM: 0.184). Both the LLM and rule-based adaptations converge toward similarly low absolute mIoU under the single-reference design.

Table 5
Cumulative mIoU contribution (mean Δ vs. previous level).

Transition	Opus-4.6	GPT-5.4	Gemini-3-Flash
Baseline → *Non-LLM*	+0.051	+0.051	+0.051
Baseline → m	−0.006	+0.032	+0.049
m → m+s	**+0.176**	**+0.117**	**+0.103**
m+s → m+s+k	+0.022	+0.021	+0.014

4.2. Ablation analysis

To isolate each context component's contribution, Table 5 reports cumulative mIoU gains at each ablation step. Fig. 9 visualises the step-wise progression for regular and complex categories across all three LLMs.

Memory alone provides an initial reference but is insufficient, and in some settings it degrades performance. On regular tunnels, especially the staggered subsets, memory alone changes mIoU by −0.031 to +0.053 across the three LLMs (i.e., approximately −10.7% to +18.3% relative to the regular-category static baseline of 0.29; Table 5; two of the three LLMs show small positive averages, while one shows a small negative average). Without intermediate feedback, the agent attempts to adapt parameters but cannot verify whether adjustments improve or degrade the pipeline outputs.

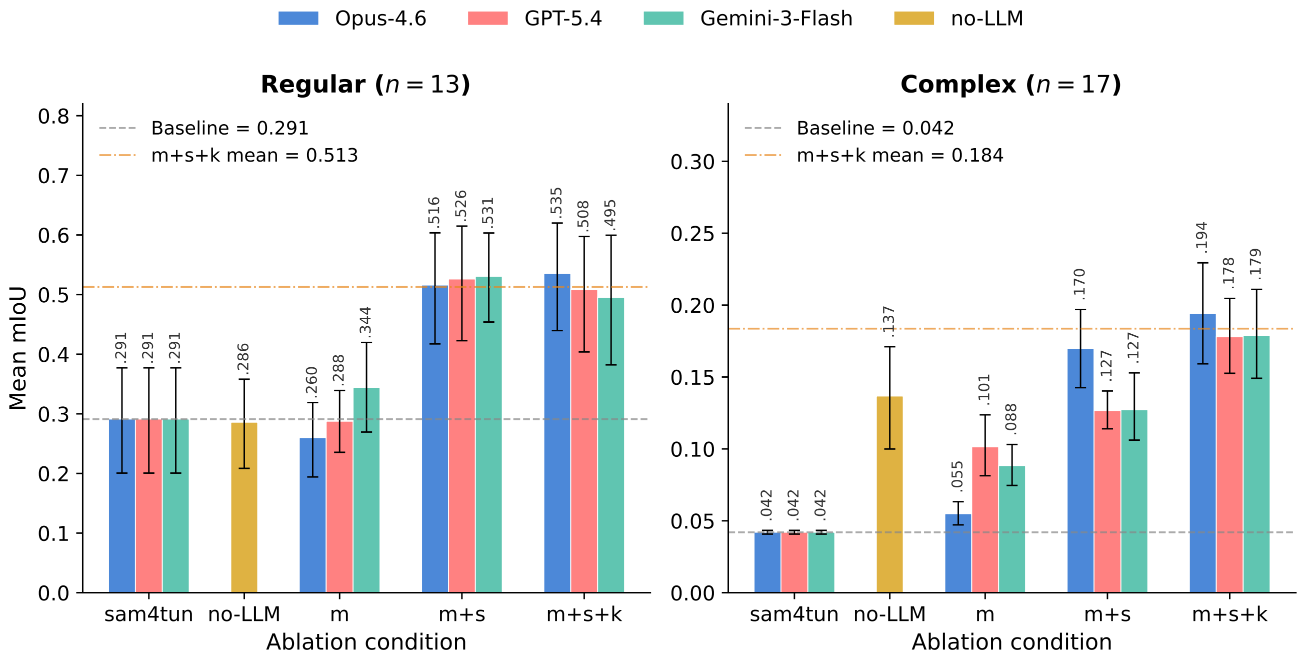


Figure 9: Step-wise adaptation results split by tunnel categories. Bars left to right: SAM4Tun (static baseline), *Non-LLM* (rule-based adaptation), m, m+s, m+s+k. The rule baseline closes part of the gap on complex tunnels but stays flat on regular tunnels; LLM (m+s+k) is highest in both categories. Error bars are bootstrap 95% CIs.

State accounts for the largest observed increment in the ablation study. The m+s condition yields the strongest gain relative to baseline across all three LLMs (all $p < 0.0001$; paired Cohen's $d = 1.32$–1.77), while the memory-only effect is small or inconsistent, indicating that state is the main contributor to the observed improvement. State provides the agent with explicit quantitative evidence of how each stage has transformed the data: radial percentiles for mask bounds, retention rates for denoising aggressiveness, coverage uniformity for upsampling targets. Without state, the agent has only pre-pipeline statistics that may not reflect conditions after unfolding, denoising, or enhancing. The same conclusion holds when the comparator is the Non-LLM baseline rather than the static SAM4Tun: state alone (m+s) lifts overall mIoU by 0.10–0.12 above the rule-based adaptation (a relative increase of 50%–60% over the rule overall mIoU of 0.20).

One plausible explanation is that the gain associated with state depends on how the model maps raw numeric summaries (percentiles, counts, ratios) to parameter values in light of parameter semantics. We did not test alternative mapping strategies (e.g., regression models); however, the continuous, multidimensional nature of the characteristic space and the non-trivial parameter interactions suggest that capturing this mapping through static rules alone would require considerable manual engineering effort per tunnel category.

Knowledge provides a small additional gain, concentrated in the complex-category. On top of m+s, knowledge raises mean mIoU by +0.014 to +0.022 across the three LLMs. The mean increment are narrow, but the per-tunnel direction is nevertheless positive. The increment is concentrated where the knowledge document supplies specific tuning guidance that neither memory nor state can reliably infer from numerical summaries alone (e.g., the coupling between ring diameter and spacing, where `ring_spacing_constant` should increase with diameter).

Table 6
Cross-model summary (m+s+k vs baseline, overall).

LLM	Mean ΔmIoU	95% CI	Cohen's d
Opus-4.6	+0.192	[0.158, 0.227]	1.95
GPT-5.4	+0.171	[0.140, 0.203]	1.95
Gemini-3-Flash	+0.166	[0.126, 0.204]	1.51

4.3. Cross-model consistency

To assess whether the adaptation is model-dependent or driven by objective tunnel conditions, Table 6 compares the three LLMs on the full m+s+k condition. The three models show similar effect ranges under the full m+s+k condition, although overlapping 95% confidence intervals do not by themselves rule out between-model differences.

4.4. Parameter sensitivity

This analysis separates parameters that vary across tunnels from those that remain constant, clarifying which aspects of the adaptation respond to specific tunnel conditions.

Analysis of 270 adapted parameter runs identified 18 parameters that all three LLMs consistently adjust (Table 7). Of these, 11 are tunnel-responsive (CV $\geq$ 0.06), in that

Table 7
Critical parameters identified across all three LLMs (18 total). *Tunnel-responsive* parameters (CV ≥ 0.06) vary with tunnel geometry; *baseline corrections* (CV ≈ 0) take near-identical values across tunnels.

Stage	Parameter	Behaviour	Tunnels	CV	Adapted range / correction
Unfolding	`diameter`	Responsive	27/30	0.072	[5.31, 7.6]
Denoising	`mask_r_low`	Responsive	30/30	0.082	[2.09, 3.75]
Denoising	`mask_r_high`	Responsive	30/30	0.147	[2.78, 4.38]
Denoising	`default_cutoff_z`	Responsive	29/30	0.142	[2.65, 6.27]
Denoising	`z_step`	Responsive	30/30	0.181	[0.003, 0.005]
Denoising	`smooth_win`	Correction	30/30	≈0	3 → 5
Denoising	`smooth_offset`	Correction	30/30	≈0	−0.003 → −0.002
Denoising	`grad_thresh`	Correction	30/30	≈0	0.2 → 0.15
Denoising	`y_step`	Correction	30/30	≈0	0.5 → 0.4
Enhancing	`inter_radius`	Responsive	30/30	0.130	[0.03, 0.08]
Enhancing	`upsampling_stage1`	Responsive	30/30	0.064	[0.055, 0.11]
Enhancing	`curv_thresh`	Correction	30/30	≈0	0.0005 → 0.005
Enhancing	`depth_low`	Correction	30/30	≈0	0.003 → 0.005
Enhancing	`depth_high`	Correction	30/30	≈0	0.008 → 0.015
Segmenting	`hough_thresh_obliq`	Responsive	30/30	0.188	[20, 83]
Segmenting	`hough_thresh_horiz`	Responsive	30/30	0.204	[20, 83]
Segmenting	`hough_thresh_vert`	Responsive	28/30	0.219	[320, 980]
Segmenting	`processing.padding`	Responsive	29/30	0.265	[160, 419]

their adapted values vary with tunnel geometry, clustering by tunnel category. The remaining 7 parameters behave as baseline corrections (CV $\approx$ 0), with near-identical values across tunnels, indicating a shared correction trend relative to the SAM4Tun defaults.

5. Discussion

5.1. Key findings

The experimental evaluation yields two primary findings. First, as noted in the ablation study (Section 4.2), the context design is shown to help adaptation across models and tunnel categories, with state providing the main gains (+0.103 to +0.176 mIoU on top of memory). Second, the LLM (m+s+k) gain over the non-LLM rule-based adaptation is concentrated on the regular-category, where the rule table cannot improve over the static baseline (Section 4.1). This concentration is due to the single-reference design: both comparators start from a single configuration tuned on a tunnel that is similar to the regular-category. Near the reference, the LLM has sufficient contextual information to reason from, and its advantage over deterministic lookup becomes visible; far from the reference (the complex-category), neither approach has a matching anchor, and the two converge toward similarly low absolute performance, and the LLMs still score higher. As reported in Section 4.1, the regular-category gap occurs where the deterministic control does not improve over the static baseline, whereas the complex-category convergence more likely reflects a shared ceiling than a specific failure of LLM reasoning (Section 5.2).

In practice, a domain expert calibrates the pipeline on a single reference tunnel, encodes the tuning rationale into structured documents, and deploys R4Tun for subsequent tunnels. Within this setup, R4Tun is most useful as an automated adaptation tool for tunnels close to the reference configuration, and can also serve as a diagnostic aid that flags where the fixed pipeline begins to fail. The framework also supports post-hoc review by logging a rationale alongside each parameter change, and requires no model retraining or labelled domain data.

5.2. Limitations

R4Tun was evaluated exclusively on the SAM4Tun pipeline and the Seg2Tunnel dataset; as a result, its transferability to other point-cloud processing pipelines and datasets requires further testing. All parameter adaptation is anchored to a single expert-tuned configuration, which creates a low ceiling for both the LLM-guided and the rule-based methods. When no contextual anchor resembles the target tunnel, neither approach has sufficient information to recover strong performance. Expanding R4Tun to encode multiple expert-tuned reference configurations (e.g., one per tunnel category), may further narrow the absolute gap. The auditability of the generated reasoning traces has not been validated through a formal user study with practising engineers. Finally, parameters are adapted in a single forward pass without iterative self-correction, leaving potential gains from closed-loop refinement unexplored.

6. Conclusions

Reliable segmentation of segmental tunnel linings from point clouds remains challenging when tunnel conditions diverge from the expert-tuned reference. This paper presented R4Tun, a multi-agent framework that extends a fixed segmentation pipeline with LLM-guided parameter adaptation.

Rather than replacing the underlying geometric operators, R4Tun adapts their parameters by comparing each new tunnel with the reference configuration and by using compact summaries of intermediate pipeline outputs. This design preserves the deterministic structure of the original pipeline while supporting post-hoc review and revision of parameter changes.

Evaluated on 30 subsets spanning 13 regular and 17 complex tunnels with three independent LLMs, the full memory + state + knowledge design raised mean mIoU from 0.15 to 0.32–0.34 (a relative increase of 113.3%–126.7%; bootstrap 95 % CIs exclude zero for all three LLMs), with per-class IoU improving broadly across structural classes for regular tunnels and recovering progressively from near-zero baselines for complex tunnels (regular: 0.29 → 0.50–0.54, +72.4%–86.2%; complex: 0.04 → 0.18–0.19, +350.0%–375.0%). The ablation study suggested that state is an important contributor to improvement (memory+state vs. baseline: paired Cohen's $d > 1.3$), with all three LLMs converging on a similar subset of critical parameters and tunnel-category adaptation patterns. A non-LLM rule-based adaptation reaches overall mIoU 0.20 (+33.3% overall) but does not improve on the regular-category static baseline (≈ 0.29); in this setup, the lift to 0.50–0.54 on the regular-category is therefore consistent with an additional LLM-related contribution. On the complex-category, the LLM and rule-based adaptations converge at low absolute mIoU, which we attribute to the single-reference design rather than to LLM reasoning quality.

For construction practice, R4Tun reallocates expert effort from per-tunnel intervention to a one-off authoring step (reference calibration plus per-stage knowledge documents), makes each parameter change auditable via a logged rationale, and runs via affordable APIs without retraining on labelled domain data.

Several directions for future work follow from the limitations above. First, encoding multiple expert-designed reference configurations could improve adaptation to atypical tunnels. Second, adding an iterative self-evaluation step based on multiple intrinsic criteria, jointly considering coverage, class balance, and geometric continuity, may capture aspects missed by a single coverage metric. Third, validation on broader tunnel typologies and acquisition conditions would strengthen generalisability claims.

Data availability

The source code, adapted parameters, and evaluation scripts are available at https://github.com/Tao-Robominds/R4Tun. The Seg2Tunnel dataset is publicly available.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Declaration of generative AI use

During this study, the authors evaluated large language models (Opus-4.6, GPT-5.4, and Gemini-3-Flash) as the main experimental systems. The LLMs were also used for language editing and for improving manuscript structure. The authors reviewed and edited all content and take full responsibility for the content of the publication.

Appendices

A. Baseline parameter tables

Tables 8–11 report the SAM4Tun baseline parameter values used as the reference configuration for all adaptation experiments.

B. Characteriser fields

Table 12 summarises the characteriser fields used to describe each tunnel and to populate the per-stage state provided to the agents.

C. Non-LLM rules-based pseudocode

The non-LLM baseline (Section 3.5) reads the same per-stage knowledge documents the LLM agents read, transcribed into a deterministic Python lookup. The parameter-selection logic for the denoising stage is reproduced below. Other stages follow the same pattern.

```
def select_denoising(chars):
    diameter = chars["estimated_diameter"]
    density  = chars["density"]
    p10      = chars["unfolded_p10"]
    p99      = chars["unfolded_p99"]

    if diameter > 6.5:
        family = "large"
    elif chars["joint_type"] == "continuous":
        family = "continuous"
    else:
        family = "base"

    params = REFERENCE_DENOISING.copy()

    if family == "large":
        params["mask_r_low"]       = p10
        params["mask_r_high"]      = p99 + 0.05
        params["smooth_win"]       = 5
    elif family == "continuous":
        params["mask_r_high"]      = max(p99, 2.85)
        params["smooth_win"]       = 6
    return params
```

D. Context components: denoising agent example

This appendix reproduces the three context components (memory, state, knowledge) that the agent receives as input.

Table 8
Stage 1 — Unfolding parameters (sam4tun baseline).

Parameter	Value
delta	0.005
slice_spacing_factor	1.2
vertical_filter_window	4.5
ransac_threshold	1
ransac_probability	0.9
ransac_inlier_ratio	0.75
ransac_sample_size	5
polynomial_degree	3
num_samples_factor	1210
diameter	5.5

Table 9
Stage 2 — Denoising parameters (sam4tun baseline).

Parameter	Value
mask_r_low	2.7
mask_r_high	2.8
y_step	0.5
z_step	0.001
grad_threshold	0.2
smoothing_window_size	3
smoothing_offset	−0.003
default_cutoff_z	2.7

Table 10
Stage 3 — Enhancing parameters (sam4tun baseline).

Parameter	Value
upsamp_stage1_target_dist	0.08
upsamp_stage2_target_dist	0.04
upsamp_stage3_target_dist	0.02
curvature_threshold	0.0005
depth_threshold_low	0.003
depth_threshold_high	0.01
inter_radius	0.06
duplicate_threshold	0.02
num_neighbors	20
num_interpolations	2
resolution	0.005
window_size	9

D.1. Memory: reference vs target raw characteristics

Memory pairs the reference tunnel's raw characteristics with those of the target tunnel using an identical schema, so the agent can compute deviations field-by-field (Table 13).

Memory also bundles the SAM4Tun reference parameters for the stage (Table 9), so the agent always has a known-good baseline to deviate from.

Table 11
Stage 4 — Segmenting parameters (sam4tun baseline). *

Parameter	Value
Boundary detection	
binary_threshold	127
morph_kernel_size	[3, 3]
dilation_iterations	1
hough_thresh_oblique	50
minLineLength_oblique	100
maxLineGap_oblique	40
hough_thresh_horiz	50
minLineLength_horiz	100
maxLineGap_horiz	10
hough_thresh_vert	500
angle_range_obliq_pos	[6, 9]
angle_range_obliq_neg	[−9, −6]
merge_distance	3
ring_spacing_constant	1.2
resolution	0.005
SAM template	
segment_per_ring	6
segment_order	[K, B1, A1, A2, A3, B2]
segment_width	1200
K_height	1079.92
AB_height	3239.77
angle	7.52
processing.padding	150
processing.y_bounds	[4200, 13100]
processing.crop_margin	50

D.2. State: cumulative outputs of upstream stages

State grows as the pipeline executes. For the denoising agent, state consists of the unfolded characteristics produced by the preceding unfolding stage, supplied as a reference/target pair (Table 14).

D.3. Knowledge: parameter semantics, ranges, and constraints

Knowledge is a stage-specific Markdown document, authored once and shared across all tunnels. It enumerates each parameter together with its empirically validated range, defaults, and inter-parameter constraints. The denoising knowledge document is reproduced verbatim below.

> **Tunable parameters.** `mask_r_low` (m), inner radial gate before depth histogramming, range [2.09, 3.75] (baseline 2.7); `mask_r_high` (m), outer radial gate, range [2.78, 4.38] (baseline 2.8); `default_cutoff_z` (m), fallback radial cutoff when a θ-bin lacks reliable counts, range [2.65, 6.27] (baseline 2.7); `z_step` (m), radial bin width per histogram column, range [0.003, 0.005] (baseline 0.001).
>
> **Proven defaults.**
>
> `smoothing_window_size` $= 5$,
>
> `smoothing_offset` $= -0.002$,
>
> `grad_threshold` $= 0.15$,
>
> `y_step` $= 0.4$.

Table 12
Characteriser fields by stage. Raw fields are available to all stages; each subsequent group becomes available after the corresponding stage completes.

Source	Fields
Raw	estimated diameter, tunnel length, tunnel height z-range (min, max) mean / median / min nearest-neighbour distance
Unfolded	r-percentiles (p_{10}, p_{99}), h-span, θ-span, θ-range median / std nearest-neighbour distance intensity median, intensity min
Denoised	mean / median / std nearest-neighbour distance estimated diameter, tunnel length, surface completeness surface regularity, average curvature, section curvatures
Enhanced	total points, median / mean nearest-neighbour distance coverage uniformity template spacing suitability, current median spacing

Table 13
Memory excerpt (raw characteristics). Reference is the expert-tuned tunnel; target is tunnel 4-1.

Field	Reference	Target (4-1)	Δ
Total points	1,109,768	1,872,537	+69%
Estimated diameter (m)	5.32	7.41	+39%
Tunnel length (m)	12.16	18.10	+49%
Tunnel height (m)	5.08	7.72	+52%
Mean NN distance (m)	0.0082	0.0081	−1%
Median NN distance (m)	0.0065	0.0065	$\approx$ 0%

Table 14
State excerpt (unfolded characteristics, after the unfolding stage).

Field	Reference	Target (4-1)
r-percentile p_{10} (m)	2.30	2.38
r-percentile p_{99} (m)	2.77	3.93
h-span (m)	12.08	18.86
θ-span (rad)	17.28	23.28
Median NN distance (m)	0.047	0.066

Locked parameters. All other keys in the stage JSON remain at the pipeline defaults reported in Table 9.

Constraints. `mask_r_low` < `mask_r_high`; `default_cutoff_z` should stay coherent with the upstream unfolding `diameter` (order of magnitude $\approx$ diameter/2, with diameter in metres).

The three other stages (unfolding, enhancing, segmenting) follow the same four-block template; the per-stage knowledge documents are released in the project's code repository.

E. Worked CoT trace

The following trace continues the worked example from Appendix D: the denoising agent receives the memory, state, and knowledge inputs above, executes the five-step CoT protocol (Section 3.3.2), and emits a single schema-conformant JSON object that the denoising stage consumes unchanged.

Step 1 — Referencing. Estimated diameter is 7.41 m vs reference 5.32 m (+39%). Unfolded r-percentiles are $p_{10} = 2.38$, $p_{99} = 3.93$, vs reference $p_{10} = 2.30$, $p_{99} = 2.77$. Median nearest-neighbour distance after unfolding is 0.066 m vs reference 0.047 m (+41%), indicating lower areal density on the tunnel wall.

Step 2 — Diagnostic inspection. Large-diameter, moderate-sparse target. The reference `mask_r_low`/`mask_r_high` (2.7/2.8) would clip all wall points beyond $r = 2.8$ m, which is below the target $p_{10} = 2.38$ and well below $p_{99} = 3.93$; the radial gate must widen substantially. `default_cutoff_z` (2.7) is below the target p_{99} (3.93) and would force the fallback path to discard valid wall points; it must rise. Lower density (+41% NN distance) requires `z_step` to relax from 0.001 so each radial bin retains enough samples for the gradient test.

Step 3 — Parameter adaptation. Set `mask_r_low` = 2.37 (just below target $p_{10} = 2.38$); `mask_r_high` = 3.80 (just below target $p_{99} = 3.93$, retaining $\geq$ 99% of wall points while staying within the empirical range [2.78, 4.38]); `default_cutoff_z` = 3.7 (close to `mask_r_high` and order $\approx$ diameter/2 = 3.71); `z_step` = 0.005 (proven default for sparse scans). All other parameters take the proven defaults from the knowledge block.

Step 4 — Validation. `mask_r_low` (2.37) < `mask_r_high` (3.80); both lie within their empirical ranges. `default_cutoff_z` (3.7) is consistent with the unfolding `diameter` (7.41 m, half-diameter 3.71 m). `z_step` (0.005) lies in [0.003, 0.005]. All keys present, all types match Table 9.

Step 5 — JSON output.


```
{
  "mask_r_low": 2.37,
  "mask_r_high": 3.80,
  "y_step": 0.4,
  "z_step": 0.005,
  "grad_threshold": 0.15,
  "smoothing_window_size": 5,
  "smoothing_offset": -0.002,
  "default_cutoff_z": 3.7
}
```

Table 15
Runtime, API calls, and per-tunnel API cost. Token counts are as reported by each vendor API; USD figures use vendor list prices at submission and are indicative.

Metric	Value
LLM API calls per tunnel	5 (one per pipeline stage)
Total API calls (full ablation)	150 per condition per LLM
Wall-clock time per tunnel	~24 min
GPU	Single NVIDIA RTX 5060
Retraining required	None
Labelled data required	None (GT for evaluation only)
Per-tunnel tokens and cost (m+s+k, mean over 30 tunnels)	
Opus 4.6	64,854 in / 6,599 out, $1.47
GPT-5.4	64,414 in / 35,263 out, $0.43
Gemini 3 Flash	83,658 in / 3,058 out, $0.03

Table 16
Per-class IoU—Regular tunnels ($n = 13$, 6-class, Opus 4.6).

Class	sam4tun	rules	memory	m+s	m+s+k
Background	0.640	0.597	0.559	0.741	0.751
K-block	0.192	0.222	0.129	0.373	0.386
B1-block	0.261	0.250	0.206	0.527	0.540
A1-block	0.253	0.263	0.276	0.537	0.542
A2-block	0.150	0.144	0.159	0.380	0.420
A3-block	0.286	0.289	0.259	0.519	0.550
B2-block	0.256	0.236	0.232	0.534	0.558

The same parameter keys are adapted by GPT-5.4 and Gemini 3 Flash in qualitatively the same direction for this tunnel.

F. Runtime, API calls, and cost

Table 15 reports the compute setup and, for one `m+s+k` run, the per-tunnel input/output token counts and indicative USD cost for each LLM (averaged over the 30 tunnels, five stage calls per tunnel).

G. Per-class IoU breakdown

Tables 16 and 17 report per-class IoU for Opus 4.6 (representative; other LLMs show the same pattern) alongside the rules baseline. For regular tunnels, rules achieve per-class IoU comparable to sam4tun while all LLM conditions improve roughly uniformly. For complex tunnels (rules $n = 17$ including 3 failed tunnels scored as zero), rules recover segment structure from near-zero for most classes but not K-block; the LLM conditions achieve further improvement on regular tunnels. B2-block remains the hardest class, requiring the 7-segment layout guidance from the knowledge component.

Table 17
Per-class IoU—Complex tunnels ($n = 17$, 7-class, Opus 4.6). Rules include 3 failed tunnels scored as zero.

Class	sam4tun	rules	memory	m+s	m+s+k
Background	0.337	0.534	0.358	0.513	0.520
K-block	0.000	0.000	0.034	0.183	0.159
B1-block	0.000	0.041	0.001	0.084	0.135
A1-block	0.000	0.072	0.005	0.128	0.155
A2-block	0.000	0.083	0.006	0.143	0.116
A3-block	0.000	0.149	0.017	0.092	0.119
A4-block	0.000	0.116	0.019	0.100	0.104
B2-block	0.000	0.099	0.000	0.000	0.043

Table 18
Performance distribution (mean across 3 LLMs).

Metric	sam4tun	rules	memory	m+s	m+s+k
Mean mIoU	0.150	0.201	0.175	0.304	0.320
Std	0.166	0.132	0.136	0.228	0.218
Min	0.032	0.000	0.042	0.082	0.072
Max	0.532	0.484	0.471	0.682	0.679

H. Performance distribution

Table 18 summarises the distribution of mIoU across tunnels (reported as the mean across the three LLMs for each condition).

References

[1] L. Attard, C. J. Debono, G. Valentino, M. D. Castro, Tunnel inspection using photogrammetric techniques and image processing: A review, ISPRS Journal of Photogrammetry and Remote Sensing 144 (2018) 180–188.

[2] L. Weidner, G. Walton, Generalized extraction of bolts, mesh, and rock in tunnel point clouds: a critical comparison of geometric feature-based methods using random forest and neural networks, Remote Sensing 16 (2024) 678.

[3] M. Q. Huang, J. NiniÄĞ, Q. B. Zhang, Bim, machine learning and computer vision techniques in underground construction: current status and future perspectives, Tunnelling and Underground Space Technology 108 (2021) 103677.

[4] A. SjÃűlander, V. Belloni, A. Ansell, E. NordstrÃűm, Towards automated inspections of tunnels: a review of optical inspections and autonomous assessment of concrete tunnel linings, Sensors 23 (2023) 5457.

[5] R. Montero, J. G. Victores, S. MartÃŋnez, A. JardÃşn, C. Balaguer, Past, present and future of robotic tunnel inspection, Automation in Construction 59 (2015) 99–112.

[6] A. Strauss, J. Bien, H. Neuner, C. Harmening, C. Seywald, M. ÃŰsterreicher, K. Voit, E. Pistone, P. Spyridis, K. Bergmeister, Sensing and monitoring in tunnels: testing and monitoring methods for the assessment of tunnels, Structural Concrete 21 (2020) 1234–1248.

[7] Y. Duan, S. Qiu, W. Jin, T. Lu, X. Li, High-speed rail tunnel panoramic inspection image recognition technology based on improved yolov5, Sensors 23 (2023) 6786.

[8] Y. J. Cha, R. Ali, J. Lewis, O. Buyukozturk, Deep learning-based structural health monitoring, Automation in Construction 161 (2024) 105324.

[9] C. Su, Q. Hu, Z. Yang, R. Huo, A review of deep learning applications in tunneling and underground engineering in china, Applied Sciences 14 (2024) 3234.

[10] R. Bommasani, D. A. Hudson, E. Adeli, R. Altman, S. Arora, S. von Arx, M. S. Bernstein, J. Bohg, A. Bosselut, E. Brunskill, E. Brynjolfsson, S. Buch, D. Card, R. Castellon, N. Chatterji, A. Chen, K. Creel, J. Q. Davis, D. Demszky, C. Donahue, M. Doumbouya, E. Durmus, S. Ermon, J. Etchemendy, K. Ethayarajh, L. Fei-Fei, C. Finn, T. Gale, K. Goel, N. Goodman, S. Grossman, et al., On the opportunities and risks of foundation models, arXiv preprint arXiv:2108.07258 (2021).

[11] A. Kirillov, E. Mintun, N. Ravi, H. Mao, C. Rolland, L. Gustafson, T. Xiao, S. Whitehead, A. C. Berg, W.-Y. Lo, P. DollÃąr, R. Girshick, Segment anything, Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV) (2023) 3992–4003.

[12] Z. Ye, W. Lin, A. Faramarzi, X. Xie, J. NiniÄĞ, Sam4tun: No-training model for tunnel lining point cloud component segmentation, Tunnelling and Underground Space Technology 158 (2025) 106401.

[13] J. Bradley, A. Bran, T. Sellam, et al., Chemcrow: Augmenting large-language models with chemistry tools, arXiv preprint arXiv:2304.05376 (2023).

[14] C. Qian, W. Liu, H. Liu, N. Chen, Y. Dang, J. Li, C. Yang, W. Chen, Y. Su, X. Cong, J. Xu, D. Li, Z. Liu, M. Sun, Chatdev: Communicative agents for software development, arXiv preprint arXiv:2307.07924 (2024).

[15] S. Hong, M. Zhuge, J. Chen, X. Zheng, Y. Cheng, C. Zhang, J. Wang, Z. Wang, S. K. S. Yau, Z. Lin, L. Zhou, C. Ran, L. Xiao, C. Wu, J. Schmidhuber, Metagpt: Meta programming for a multi-agent collaborative framework, arXiv preprint arXiv:2308.00352 (2024).

[16] P. Chen, B. Han, S. Zhang, Comm: Collaborative multi-agent, multi-reasoning-path prompting for complex problem solving, arXiv preprint arXiv:2405.00847 (2024).

[17] C. I. Garcia, M. A. Dibattista, T. A. Letelier, H. D. Halloran, J. A. Camelio, Framework for llm applications in manufacturing, Manufacturing Letters 40 (2024) 56–63.

[18] M. A. Fischler, R. C. Bolles, Random sample consensus: a paradigm for model fitting with applications to image analysis and automated cartography, Communications of the ACM 24 (1981) 381–395.

[19] M. Ester, H. P. Kriegel, J. Sander, X. Xu, A density-based algorithm for discovering clusters in large spatial databases with noise, Proceedings of the Second International Conference on Knowledge Discovery and Data Mining (KDD) (1996) 226–231.

[20] M. Pauly, M. Gross, L. P. Kobbelt, Efficient simplification of point-sampled surfaces, Proceedings of IEEE Visualization (2002) 163–170.

[21] C. R. Qi, L. Yi, H. Su, L. J. Guibas, Pointnet++: Deep hierarchical feature learning on point sets in a metric space, in: Advances in Neural Information Processing Systems (NeurIPS), 2017. URL: https://arxiv.org/abs/1706.02413.

[22] Q. Hu, B. Yang, L. Xie, S. Rosa, Y. Guo, Z. Wang, N. Trigoni, A. Markham, Randla-net: Efficient semantic segmentation of large-scale point clouds, Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR) (2020).

[23] J. Schult, F. Engelmann, A. Hermans, O. Litany, S. Tang, B. Leibe, Mask3d: Mask transformer for 3d semantic instance segmentation, arXiv preprint arXiv:2303.05475 (2023).

[24] M. Kolodiazhnyi, A. Vorontsova, A. Konushin, D. Rukhovich, Top-down beats bottom-up in 3d instance segmentation, Proceedings of the IEEE/CVF Winter Conference on Applications of Computer Vision (WACV) (2024) 3566–3574.

[25] E. Camuffo, D. Mari, S. Milani, Recent advancements in learning algorithms for point clouds: an updated overview, Sensors 22 (2022) 1357.

[26] M. Liu, R. Shi, K. Kuang, Y. Zhu, X. Li, S. Han, H. Cai, F. Porikli, H. Su, Openshape: Scaling up 3d shape representation towards open-world understanding, 2023. URL: https://arxiv.org/abs/2305.10764.

[27] Z. Guo, R. Zhang, X. Zhu, Y. Tang, X. Ma, J. Han, K. Chen, P. Gao, X. Li, H. Li, P.-A. Heng, Point-bind & point-llm: Aligning point cloud with multi-modality for 3d understanding, generation, and instruction following, arXiv preprint arXiv:2309.00615 (2023).

[28] R. Xu, X. Wang, T. Wang, Y. Chen, J. Pang, D. Lin, Pointllm: Empowering large language models to understand point clouds, in: Proceedings of the European Conference on Computer Vision (ECCV), 2024, pp. 131–147.

[29] M. Caron, H. Touvron, I. Misra, H. Jégou, J. Mairal, P. Bojanowski, A. Joulin, Emerging properties in self-supervised vision transformers, arXiv (2021). Version 2, revised 24 May 2021.

[30] G. Franceschelli, C. Cevenini, M. Musolesi, Training foundation models as data compression: on information, model weights and copyright law, arXiv preprint arXiv:2501.04023 (2025).

[31] N. Ravi, V. Gabeur, Y.-T. Hu, R. Hu, C. Ryali, T. Ma, H. Khedr, R. RÃďdle, C. Rolland, L. Gustafson, E. Mintun, J. Pan, K. V. Alwala, N. Carion, C.-Y. Wu, R. Girshick, P. DollÃąr, C. Feichtenhofer, Sam 2: Segment anything in images and videos, arXiv preprint arXiv:2408.00714 (2024).

[32] X. Zou, J. Yang, H. Zhang, F. Li, L. Li, J. Wang, L. Wang, J. Gao, Y. J. Lee, Segment everything everywhere all at once, arXiv preprint arXiv:2304.06718 (2023).

[33] R. R, S. S, N. V. Kumar, R. S, P. B. V, Crack-sam: Crack segmentation using a foundation model, arXiv preprint arXiv:2401.15201 (2024).

[34] B. Wang, Z. Chen, M. Li, Q. Wang, C. Yin, J. C. P. Cheng, Omni-scan2bim: A ready-to-use scan2bim approach based on vision foundation models for mep scenes, Automation in Construction 167 (2024) 105678.

[35] F. Pan, S. Jeon, B. Wang, F. McKenna, S. X. Yu, Zero-shot building attribute extraction from large-scale vision and language models, arXiv preprint arXiv:2312.12479 (2023).

[36] L. Ouyang, J. Wu, X. Jiang, D. Almeida, C. L. Wainwright, P. Mishkin, C. Zhang, S. Agarwal, K. Slama, A. Ray, J. Schulman, J. Hilton, F. Kelton, L. Miller, M. Simens, A. Askell, P. Welinder, P. Christiano, J. Leike, R. Lowe, Training language models to follow instructions with human feedback, arXiv preprint arXiv:2203.02155 (2022).

[37] DeepSeek-AI, DeepSeek-R1: Incentivizing reasoning capability in LLMs via reinforcement learning, arXiv preprint arXiv:2501.12948 (2025).

[38] J. Chua, O. Evans, Are deepseek r1 and other reasoning models more faithful?, arXiv preprint arXiv:2501.07632 (2025).

[39] V. Xiang, C. Snell, K. Gandhi, A. Albalak, A. Singh, C. Blagden, D. Phung, R. Rafailov, N. Lile, D. Mahan, L. Castricato, J.-P. Fränken, N. Haber, C. Finn, Towards System 2 reasoning in LLMs: Learning how to think with meta chain-of-thought, arXiv preprint arXiv:2501.04682 (2025).

[40] OpenAI, OpenAI o3 and o4-mini system card, https://openai.com/index/o3-o4-mini-system-card/, 2025. Accessed: 2025-12-01.

[41] OpenAI, Reasoning best practices, https://platform.openai.com/docs/guides/reasoning-best-practices, 2025. Accessed: 2025-12-01.

[42] J. Wei, X. Wang, D. Schuurmans, M. Bosma, B. Ichter, F. Xia, E. Chi, Q. Le, D. Zhou, Chain-of-thought prompting elicits reasoning in large language models, arXiv preprint arXiv:2201.11903 (2023).

[43] T. Kojima, S. S. Gu, M. Reid, Y. Matsuo, Y. Iwasawa, Large language models are zero-shot reasoners, arXiv preprint arXiv:2205.11916 (2023).

[44] Z. Zhang, Y. Yao, A. Zhang, X. Tang, X. Ma, Z. He, Y. Wang, M. Gerstein, R. Wang, G. Liu, H. Zhao, Igniting language intelligence: the hitchhiker's guide from chain-of-thought reasoning to language agents, arXiv preprint arXiv:2311.11797 (2023).

[45] Y. Zhang, R. Sun, Y. Chen, T. Pfister, R. Zhang, S. Ã. Arik, Chain of agents: Large language models collaborating on long-context tasks, arXiv preprint arXiv:2406.02818 (2024).

[46] P. Xu, W. Ping, X. Wu, L. McAfee, C. Zhu, Z. Liu, S. Subramanian, E. Bakhturina, M. Shoeybi, B. Catanzaro, Retrieval meets long context large language models, arXiv preprint arXiv:2310.03025 (2024).

[47] L. Mei, J. Yao, Y. Ge, Y. Wang, B. Bi, Y. Cai, J. Liu, M. Li, Z.-Z. Li, D. Zhang, C. Zhou, J. Mao, T. Xia, J. Guo, S. Liu, A survey of context engineering for large language models, arXiv preprint arXiv:2501.04567 (2025).

[48] Anthropic, Effective context engineering for ai agents, https://www.anthropic.com/engineering/effective-context-engineering-for-ai-agents, 2025. Accessed: 2025-12-01.

[49] R. O. Duda, P. E. Hart, Use of the hough transformation to detect lines and curves in pictures, Communications of the ACM 15 (1972) 11–15.

[50] OpenAI, Gpt-4o system card, arXiv preprint arXiv:2410.21276 (2024).